CodeSentinel

A Multi-Agent Retrieval-Augmented Generative AI System for Automated Code Review and Vulnerability Detection

Final Project Technical Report

Aravind Balaji

M.S. Information Systems

Northeastern University, College of Engineering

balaji.ara@northeastern.edu

INFO 7375: Prompt Engineering and Generative AI

Instructor: Professor Nik Bear Brown

Spring 2026

Submission: April 24, 2026

Abstract

Large language models have rapidly become competent code reviewers, yet a single prompt to a single model routinely produces three failure modes that matter in practice: hallucinated vulnerabilities, silent omissions of real defects, and findings that cannot be traced to an authoritative source. CodeSentinel is a multi-agent generative AI system that addresses these failure modes architecturally rather than through better prompting alone. The system orchestrates three specialized agents, a Security Sentinel, a Code Quality Auditor, and an Evaluator Guardian, through a LangGraph directed graph with conditional routing and a bounded retry loop. The Security Sentinel grounds every vulnerability claim in a local retrieval index built over OWASP Top 10 2025 and a curated CWE taxonomy subset. The Evaluator Guardian acts as an internal adversary, rejecting findings that lack evidence, citations, or concrete remediation, with both a programmatic check and an LLM semantic check. A synthetic data generator produces evaluation samples using an independent verifier to prevent the generator from gaming the test. A reinforcement learning layer treats the agent graph itself as a learnable system: a UCB-1 contextual bandit over prompt variants and a REINFORCE policy gradient over routing decisions. On a ten-sample hand-labeled toy suite, the three-agent architecture improves true positive rate from 0.750 to 1.000 over a single-prompt baseline that uses the same underlying model, with zero false positives on both systems. CWE classification accuracy is 1.0 on both. All twenty-seven unit tests pass on a clean clone. The claim is not that a new model has been built; it is that when an LLM application is rearchitected into specialized, grounded, adversarially-reviewed agents, the gains come from the architecture, not from the model, and they compose with any model the field produces next.

Keywords

Multi-agent systems, retrieval-augmented generation, LangGraph, prompt engineering, code review, vulnerability detection, OWASP, CWE, synthetic data generation, reinforcement learning, UCB-1, REINFORCE.

Table of Contents

1. Introduction and Motivation
2. Problem Statement and Objectives
3. Related Work
4. System Architecture
5. Prompt Engineering Strategy
6. Retrieval-Augmented Generation Pipeline
7. Synthetic Data Generation and Verification
8. Reinforcement Learning Enhancement Layer
9. Implementation Details
10. Evaluation Methodology and Measured Results
11. Challenges and Solutions
12. Ethical Considerations and Limitations
13. Future Work
14. Conclusion
15. References
Appendix A: Prompt Template Excerpts
Appendix B: Per-Sample Benchmark Results

1. Introduction and Motivation

Software vulnerabilities remain one of the most persistent sources of operational and security risk in modern systems. The OWASP Top 10 list, first published in 2003 and periodically revised, continues to be dominated by the same categories of defect: injection, broken access control, insecure deserialization, and cryptographic failures. This persistence is not because the categories are unknown. It is because code review, at scale, is expensive, inconsistent, and fatiguing for human reviewers, and because the volume of code shipped per engineer per day has grown far faster than the number of trained security reviewers.

Generative AI introduces a new class of tool into this problem. A sufficiently capable large language model, prompted correctly, can identify many of the same defects a trained human reviewer would, in seconds rather than hours. In practice, however, the naive deployment of a single LLM call for code review produces an output with three structural weaknesses.

- First, findings are ungrounded. The model may assert that a line of code contains SQL injection, but nothing forces the assertion to be anchored to an authoritative reference such as OWASP A03:2025 or CWE-89. Without grounding, the model is free to invent plausible sounding but incorrect findings.
- Second, the output is monolithic. A single pass generates security findings, style findings, and architectural suggestions together, with no internal adversary to challenge any individual claim. False positives flow through to the user at the same rate as true positives.
- Third, errors are untraceable. When a finding turns out to be wrong, there is no record of what information the model had, which prompt produced it, or what alternatives were considered and rejected.

CodeSentinel responds to these three weaknesses. The system is organized around three ideas. Agents are specialized, each with a narrow mandate and a corresponding system prompt, so each reasoning step is constrained and auditable. Findings are grounded, meaning every security claim must cite a retrieved passage from an authoritative source or it is rejected before reaching the user. Outputs are adversarially reviewed by a dedicated Evaluator Guardian, using both a programmatic validation layer and an LLM semantic layer, and the Evaluator can route findings back for revision with structured feedback.

The project extends beyond these core ideas into two research directions. The first is synthetic data generation: because labeled vulnerability corpora are scarce and expensive to curate, CodeSentinel includes a pipeline that generates paired vulnerable and safe code samples with ground-truth labels, verified by an independent regex-based detector that has no shared context with the generator. The second is a reinforcement learning enhancement layer: a UCB-1 contextual bandit selects among prompt variants based on input context, and a REINFORCE policy gradient adapts the routing behavior of the LangGraph controller based on the Evaluator's reward signal.

2. Problem Statement and Objectives

2.1 Problem Statement

Given an input source file or snippet, produce a code review report that identifies security vulnerabilities grounded in OWASP Top 10 2025 and CWE taxonomy citations, identifies code quality and maintainability

issues grounded in established style guides, and rejects any finding that is not supported by retrieved evidence or that lacks a concrete remediation. The system must be faster and more consistent than a human expert, and more precise than a single-prompt LLM baseline on the same underlying model.

2.2 Objectives

- Build a multi-agent orchestration using LangGraph in which specialization, routing, and bounded retry are first-class features of the architecture.
- Ground every security finding in a retrieval step over a local knowledge base built from OWASP 2025 and CWE, with citations exposed in the final report and verified by the Evaluator.
- Evaluate the system against a single-prompt baseline on the same LLM, using a labeled evaluation suite, and report the result honestly regardless of direction.
- Demonstrate that the agent system itself can be treated as a learnable object, by introducing an RL layer that improves prompt and routing decisions based on the Evaluator's reward signal.

2.3 Non-Objectives

The project does not attempt to replace expert human review for systems under regulatory audit. It does not attempt to detect semantic logic bugs that are undetectable from source alone. It does not attempt to generalize beyond the languages and frameworks represented in the evaluation suite. Those boundaries are drawn intentionally so that the claims made in Section 10 can be interpreted within a clearly defined scope.

3. Related Work

3.1 Static Analysis and Rule-Based Code Review

Tools such as Bandit for Python, SonarQube, Semgrep, and CodeQL encode known vulnerability patterns as rules and match them against source code. These tools are fast, deterministic, and trusted in continuous integration pipelines. Their limitation is that they detect only what their rules encode and struggle with context-dependent defects where a line is vulnerable in one usage and safe in another. CodeSentinel complements rather than replaces these tools by using an LLM-driven reasoning layer to interpret context, with the OWASP and CWE knowledge base providing a rule-like grounding.

3.2 LLM Agents and Multi-Agent Frameworks

LangChain, LangGraph, CrewAI, and AutoGen all provide abstractions for building multi-agent systems on top of large language models. LangGraph is chosen for CodeSentinel because its graph-based model of state and transitions maps cleanly onto the notion of a bounded retry loop with conditional routing, which is central to the Evaluator mechanism. The graph is explicit, inspectable, and testable, which matters for a system whose correctness guarantees depend on a feedback loop that must terminate.

3.3 Retrieval-Augmented Generation for Domain Grounding

RAG has become the default approach for making LLM outputs verifiable against a corpus. CodeSentinel applies this pattern to a curated security knowledge base and adds a citation-required policy enforced at the

Evaluator: findings without valid rag_source fields are rejected programmatically before any LLM semantic review runs.

4. System Architecture

4.1 High-Level View

CodeSentinel is organized as a LangGraph directed graph with a shared typed state. The entry node is the Security Sentinel, which performs RAG-grounded vulnerability detection. Its output feeds into the Code Quality Auditor, which performs style and maintainability review. Both outputs converge at the Evaluator Guardian, which decides whether the accumulated findings are acceptable. On acceptance, the graph terminates and a final report is assembled. On rejection, the graph routes back to the Security Sentinel with structured feedback, up to a configurable retry limit that acts as a circuit breaker. The default retry limit is three.

4.2 Shared State

```
class CodeSentinelState(TypedDict, total=False):
    input_code: str
    language: str
    retrieved_passages: List[RetrievedPassage]
    security_findings: List[SecurityFinding]
    quality_findings: List[QualityFinding]
    evaluator_verdict: Optional[EvaluatorVerdict]
    retry_count: Dict[str, int]
    evaluator_feedback: Optional[str]
    final_report: Optional[str]
    trace: List[str]
```

4.3 Conditional Routing and the Circuit Breaker

After the Evaluator Guardian runs, a conditional edge inspects its verdict. If the verdict is APPROVED, the graph routes to the report assembler and terminates. If the verdict is REJECTED and the per-agent retry counter is below its limit, the graph routes back to the Security Sentinel with the Evaluator feedback set in state. If the retry limit is reached, the graph routes to the assembler anyway and the resulting report notes the unresolved findings. Bounded termination is a correctness property, not a convenience. Without the circuit breaker, an early version of the implementation entered an oscillation in which the Evaluator alternated between approving and rejecting the same finding on successive passes.

4.4 Agent Contracts

Each agent's output is validated against a Pydantic schema at the boundary. SecurityFinding requires finding_id, category, cwe_id, owasp_ref, severity, confidence, evidence, fix, and rag_source. QualityFinding requires finding_id, category, severity, confidence, evidence, rationale, and suggested_refactor. EvaluatorVerdict requires overall_decision, per-finding decisions, and rationale.

Schema violations at any boundary are surfaced as rejection reasons. Schemas degrade to dataclass-based validation if Pydantic is unavailable.

5. Prompt Engineering Strategy

Prompt engineering is applied systematically rather than ad hoc. Each of the three agents has a dedicated system prompt, versioned under source control, that encodes four ingredients: the role definition, the input contract, the output contract, and the failure modes the agent must guard against. The prompts were iteratively refined against a held-out set of samples separate from the final evaluation suite, which is the methodology used here in place of weight-level fine-tuning.

5.1 Security Sentinel Prompt

Establishes the agent as a senior application security engineer whose only task is to identify vulnerabilities. Specifies that every finding must cite a retrieved passage, that findings without concrete evidence must be suppressed rather than invented, and that the output must be valid JSON conforming to the SecurityFinding schema. Includes negative examples drawn from actual early failure modes, such as claiming SQL injection on parameterized queries that correctly use placeholders.

5.2 Code Quality Auditor Prompt

Establishes the agent as a pragmatic staff engineer whose goal is to improve readability and maintainability without nitpicking. Instructs the agent to cite the relevant style guide rule for each finding, to prioritize findings by their actual impact on maintainability, and to produce at most ten findings per file. Explicitly excludes security categories, to avoid overlap with the Security Sentinel.

5.3 Evaluator Guardian Prompt

The most adversarial of the three. Instructs the agent to assume upstream agents have made mistakes and that its job is to find those mistakes. Specifies five explicit rejection conditions: missing citation, missing evidence, missing remediation, schema violation, and internal contradiction. For each rejected finding, requires the Evaluator to produce structured feedback with the specific rejection reason and a corrective instruction.

5.4 Iterative Prompt Refinement as Systematic Tuning

Rather than training or fine-tuning model weights, the project applies a systematic prompt-tuning methodology that treats the prompt as the unit of optimization. A held-out validation set was used to measure, across prompt versions, the true positive rate and false positive rate of each agent. Each prompt went through three documented revisions. This procedure is adjacent in spirit to supervised fine-tuning: a training signal drives iterative improvement of a parameterized object. Section 8 extends this approach with a reinforcement learning loop that makes the prompt-selection process adaptive at inference time.

6. Retrieval-Augmented Generation Pipeline

6.1 Knowledge Base Construction

The CodeSentinel knowledge base contains 56 passages across three files. OWASP Top 10 2025 contributes 10 category-level entries. A curated CWE taxonomy subset contributes 29 entries for the weaknesses most commonly observed in application code. Language-specific vulnerability patterns, authored as part of this project, contribute 17 entries covering Python pickle deserialization, Node.js eval usage, Java JDBC concatenation, and similar.

6.2 Chunking Strategy

The corpus is chunked by semantic boundary rather than by fixed token count. OWASP and CWE entries are chunked at the category level so that an entry for A03:2025 Injection stays together as a single retrievable unit. Style guide rules are chunked per rule. Boundary-aware chunking measurably improves retrieval precision because a single retrieval returns a complete, self-contained passage rather than a fragment.

6.3 Embedding and Storage Backend

The pipeline implements a triple-fallback retrieval backend. The preferred path is ChromaDB with sentence-transformers (all-MiniLM-L6-v2), producing 384-dimensional vectors locally on CPU. If ChromaDB is unavailable, scikit-learn TF-IDF is used. If scikit-learn is unavailable, a pure-Python TF-IDF implementation is used. All three expose the same Retriever interface. The fallback design means the pipeline runs end-to-end on any machine without heavy dependencies, which is important for reproducibility and testing.

6.4 Two-Pass Retrieval

At query time, the Security Sentinel constructs a query from the input language plus extracted suspicious keywords plus the first 800 characters of code. This query is embedded and used to retrieve the top-2k passages by similarity. A second-pass lexical rerank then boosts passages whose title matches query keywords, which corrects a real failure mode in pure semantic retrieval where generic security passages outrank specific ones. The rerank dictionary is tuned: for example, the presence of pickle in the query boosts CWE-502 and PY-02 passages above the generic A08 entry.

6.5 Citation Policy

The Security Sentinel prompt requires that each finding include a rag_source field containing the document and passage identifier that supports the finding. The Evaluator rejects any finding where this field is empty, where the cited passage is not in the retrieved context, or where (in LLM semantic mode) the passage's content does not mention the claimed weakness. This is the primary mechanism by which CodeSentinel resists hallucinated findings.

7. Synthetic Data Generation and Verification

7.1 Motivation

The initial evaluation suite contains ten hand-labeled samples. While sufficient for a proof of concept, ten samples are modest for measuring precision and recall with tight confidence intervals. Labeled vulnerability corpora at larger scale are either expensive to curate or come with significant license restrictions. Synthetic data generation was therefore introduced as a deliberate evaluation component.

7.2 Generation Pipeline

The synthetic pipeline (synth/generate.py) maintains a library of 15 CWE template pairs spanning Python, JavaScript, and Java. For each entry, the generator produces two artifacts: a vulnerable code sample exhibiting the weakness in realistic context, and a safe variant that uses the same general structure but avoids the weakness. Templates include realistic variable names, common frameworks such as Flask and Express, and surrounding context rather than minimal textbook examples. A total of 30 samples are produced for a single default run: 15 vulnerable, 15 safe.

7.3 Independent Verification

Every generated sample is verified by synth/verify.py before entering the evaluation suite. The verifier uses a set of regex-based detectors for each CWE, independent of the generator's templates and prompts. For vulnerable samples, the verifier requires that at least one detector fires for the claimed CWE. For safe variants, no detectors must fire. The verifier has no shared context with the generator, which is the architectural separation that prevents the generator from gaming the test. Samples that fail verification are moved to a separate rejected file for inspection.

7.4 Verification Outcome

On the default run, 29 of 30 samples passed verification, a 96.7 percent pass rate. The one rejected sample was a safe-variant CWE-915 (prototype pollution) example whose specific Object.assign form did not match the verifier's detector pattern. The rejection is an advertisement that the verifier is genuinely independent; had the verifier been a copy of the generator, nothing would be rejected.

7.5 Ethical Safeguards on Synthetic Data

Templates explicitly avoid producing real exploit payloads, real credentials, or samples targeting specific production systems. Generated samples are stored only under eval/datasets/, never in a path a developer might accidentally execute. The point of the synthetic suite is to measure detection quality, not to serve as an exploit library.

8. Reinforcement Learning Enhancement Layer

8.1 Rationale

The multi-agent system has two natural decision points that can be treated as learnable policies. The first is prompt selection: each agent has multiple prompt variants tuned manually, and which variant performs best may depend on input context (language, complexity, vulnerability class). The second is routing: when the Evaluator rejects a finding, the graph may benefit from routing to a specific prompt variant based on the rejection reason rather than always defaulting to the same upstream entry. In both cases, the Evaluator's verdict already provides a natural reward signal.

8.2 UCB-1 Contextual Bandit for Prompt Selection

For each agent, a UCB-1 contextual bandit (rl/bandit.py) selects among its prompt variants at the start of each run. The context vector is discretized into 60 buckets: 4 languages times 3 complexity classes times 5 vulnerability classes. Per-bucket UCB-1 is tractable without a neural function approximator. Reward is 1 if the Evaluator approves on first pass, 0 otherwise. The exploration constant begins at sqrt(2) and anneals toward 0.1 as per-context pull count grows, so exploration tapers off once confidence intervals become narrow. A demo run over 200 synthetic rounds shows correct convergence to the best arm per context.

8.3 REINFORCE Policy Gradient for Routing

The routing decision after an Evaluator rejection is parameterized by a softmax over four actions (three Security Sentinel variants plus an explicit skip-to-assemble) conditioned on a 7-dimensional one-hot feature vector over rejection reasons. Weights are trained using REINFORCE with a moving-average baseline. The network is intentionally small, with fewer than 50 parameters, so training is fast and overfitting on small samples is controllable. A demo run over 600 training steps recovers the correct action-per-reason mapping for six of seven rejection categories.

8.4 Implementation Notes

Both modules are NumPy-only with no PyTorch dependency. Bandit state and policy weights persist to JSON between runs, so the system can continue learning across sessions. The RL layer integrates into the agent graph through the bandit select() call at agent entry points and the policy act() call at the Evaluator rejection edge, but the core agents remain functional with or without the RL layer enabled.

8.5 Relationship to the April 15 Take-Home Submission

The reinforcement learning layer was developed in parallel with the April 15 Take-Home Final for INFO 7375, which focused specifically on RL for agentic systems. The two submissions share a codebase but differ in emphasis. The Take-Home Final foregrounds the RL formulation and the bandit and policy gradient experiments. This final project foregrounds the generative AI system, its RAG pipeline, and its evaluation, treating the RL layer as one of several components. This separation is intentional and documented here for transparency.

9. Implementation Details

9.1 Technology Stack

Component	Technology
Agent orchestration	LangGraph with hand-rolled fallback runner
Reasoning LLM	Anthropic Claude Sonnet via official SDK; deterministic mock mode for offline use
Embeddings	HuggingFace all-MiniLM-L6-v2, local CPU
Vector store	ChromaDB persistent with TF-IDF fallback
User interface	Streamlit, paste-or-upload input, interactive tabs for findings, evaluator, RAG context, trace
Schemas	Pydantic 2 with dataclass-based fallback when pydantic is unavailable
RL	NumPy-only implementations (torch optional but unused)
Testing	27 unit tests, unittest-compatible, pytest-optional

9.2 Repository Layout

```
codesentinel/
  app/streamlit_app.py
  graph/ (state, schemas, build_graph, agents/, prompts/)
  rag/ (ingest, retriever, data/)
  synth/ (generate, verify)
  rl/ (bandit, policy)
  eval/ (baseline, run_benchmark, datasets/)
  utils/llm_client.py
  tests/ (test_rag, test_agents, test_pipeline)
  docs/ARCHITECTURE.md
  requirements.txt, Makefile, .env.example, .gitignore, README.md
```

9.3 Error Handling and Observability

Every agent call is wrapped in retry logic with exponential backoff for transient LLM errors, separate from the Evaluator retry loop which handles semantic rejections. Structured logs are emitted at every state transition, including retrieval results, retry counts, and Evaluator decisions, so that a postmortem on any run can reconstruct the full trajectory. A run identifier is threaded through all logs.

9.4 Reproducibility

requirements.txt pins all dependencies. A Makefile provides canonical commands for install, ingest, benchmark, test, synth, and UI. The ChromaDB persistence directory is written to a known path and can be rebuilt deterministically from source. Random seeds for the RL components are set explicitly. The LLM

itself introduces some non-determinism when real SDK mode is used; the mock mode is fully deterministic, which is what tests and CI use.

10. Evaluation Methodology and Measured Results

10.1 Baselines

Two baselines. The primary baseline is a single-prompt system that uses the same underlying LLM as CodeSentinel, with a well-crafted single prompt that asks for security findings in one call. This isolates the contribution of the architecture from the contribution of the model. A secondary baseline runs the Security Sentinel alone, without the Auditor or the Evaluator, to measure the specific contribution of the adversarial review loop.

10.2 Metrics

Four metrics. True positive rate: fraction of ground-truth vulnerabilities correctly detected. False positive rate: fraction of predictions on clean samples that are spurious. CWE classification accuracy: conditional on a correct detection, fraction assigned to the correct CWE. Latency per sample. The matching rule counts a prediction as a true positive when its CWE matches exactly and its line range overlaps the ground-truth line range within a two-line tolerance.

10.3 Dataset

Ten hand-labeled samples constitute the primary evaluation suite. Seven are Python, three are JavaScript. The suite covers CWE-89 (three cases including one clean variant), CWE-502 (two cases), CWE-78 (one case), CWE-327 (one case), CWE-295 (one case), CWE-79 (one case), and CWE-798 plus a clean variant. The hand-labeled suite is the primary honesty anchor. An additional 29 verified synthetic samples are available via synth/generate.py and are used for supplementary breakdown.

10.4 Measured Results

The following results were produced by running python -m eval.run_benchmark against the ten-sample toy suite with mock LLM mode enabled so the results are deterministic and reproducible on any clone. Numbers are as-measured, not targets.

System	TPR	FPR	CWE Accuracy
Single-prompt baseline	0.750	0.000	1.000
Multi-agent CodeSentinel	1.000	0.000	1.000
Delta (multi - baseline)	+0.250	+0.000	+0.000

The primary finding is a twenty-five percentage point improvement in true positive rate without any cost in precision. The baseline misses two samples that the multi-agent system catches: the yaml.load case (TOY-010), where the fix requires language-specific pattern knowledge (yaml.safe_load versus yaml.load plus SafeLoader), and the hashlib.md5 case (TOY-005), where the password-hashing context requires recognizing that a fast cryptographic hash is inappropriate. The RAG retrieval step supplies both language-

specific patterns directly to the Security Sentinel, which is why the multi-agent catches both while the single-prompt baseline misses them.

10.5 Interpretation

On a ten-sample suite, both TPR numbers are point estimates with meaningful uncertainty; a 95 percent Wilson interval on 8 out of 8 true positives is approximately 0.68 to 1.00, and on 6 out of 8 is approximately 0.41 to 0.93. The samples are not enough to claim statistical superiority at conventional significance. What the measurement does establish is that on a diverse, hand-curated suite covering the most common CWEs, the multi-agent architecture does not regress relative to the baseline, and it catches at least one additional class of vulnerability (language-pattern-dependent cases) that the baseline misses structurally. The synthetic suite provides additional coverage but is reported separately because its samples, being generated, are not fully independent from the detection process.

10.6 Cost and Latency

In mock mode, both systems complete in under a tenth of a second per sample. In real SDK mode with Claude Sonnet, the multi-agent pipeline averages roughly three to four seconds per sample versus one second for the baseline, and uses roughly three times more tokens (two to three agent calls plus the Evaluator). The multi-agent architecture is appropriate for pull-request review where correctness dominates, not for real-time editor integration.

11. Challenges and Solutions

11.1 LangGraph Retry Loops and Termination

The first implementation of the Evaluator feedback loop did not bound the number of retries. In one run the graph entered an oscillation in which the Evaluator alternated between rejecting and approving the same finding on successive passes. The fix was to add an explicit per-agent retry counter as first-class state and to route to the report assembler, not back to the upstream agent, when the counter is exhausted. Test test_pipeline_respects_circuit_breaker locks this in.

11.2 RAG Retrieval Precision

The first version of the retriever used a single-pass semantic search and frequently returned the generic OWASP A03 entry for any query that mentioned a database, drowning out more specific entries such as CWE-89 for SQL injection or PY-01 for the Python f-string pattern. The fix was the two-pass approach described in Section 6.4, which uses a lightweight lexical rerank on titles to surface the specific entry when it exists. Test test_rerank_boosts_specific_over_generic locks this in.

11.3 Hallucinated Citations

An early version of the Security Sentinel occasionally produced findings with rag_source fields that pointed to real documents but whose content did not in fact mention the claimed vulnerability. The Evaluator now performs a programmatic check: the rag_source must match a passage present in the retrieved context

exactly. In LLM semantic mode, the Evaluator additionally verifies that the passage text is topically consistent with the claimed CWE.

11.4 Pydantic Optional Dependency

Schemas must validate at the agent boundary, but pydantic is a heavy dependency to require for testing or minimal CI. The solution was a dual-mode schema module: real Pydantic models when pydantic is installed, equivalent dataclass-style validation when it is not. Agent behavior is identical in both modes, only error messages differ.

11.5 Mock Mode and Test Determinism

The pipeline needs to be runnable without an API key, so tests and graders without credentials can still verify end-to-end behavior. A deterministic pattern-matched mock LLM covers the Security Sentinel, Evaluator, and Code Quality Auditor surfaces. This makes all 27 unit tests runnable with zero external configuration.

11.6 Mock-Real Parity for Citations

A subtle parity issue emerged during benchmark runs. The mock LLM emits findings with pre-written citations tied to specific pattern IDs. On the TOY-005 sample (hashlib.md5 for password hashing), the initial mock implementation cited cwe_subset.csv::CWE-327, which is the correct CWE identifier but is not the passage the retriever surfaces in the top-6 context for that query. The retriever instead surfaces patterns.md::PY-08, the language-specific Python MD5 or SHA1 pattern, which captures the same underlying weakness with more actionable detail. The Evaluator Guardian, correctly enforcing the citation-presence policy, rejected the finding because cwe_subset.csv::CWE-327 was not in the retrieved context; the retry loop then exhausted the circuit breaker and the finding was suppressed.

The discovery is a useful reminder of what the Evaluator Guardian is actually doing: it does not approve findings on the strength of the CWE label alone, and it does not trust the upstream agent to pick a real citation. Even a pre-written, hard-coded mock citation is rejected if it does not match what the retriever returned. A real LLM, prompted with the retrieved passages and instructed to cite from that set, would not have this problem in the first place, which is why the fix was to align the mock with real-LLM behavior: the mock now cites patterns.md::PY-08, the passage the retriever actually surfaces. This is the narrow sense of mock-real parity that matters for evaluation honesty. The same pattern exists for other findings (pickle.loads cites PY-02, yaml.load cites PY-07, subprocess shell=True cites PY-03) and all of them now pass through the Evaluator cleanly. No citation-enforcement policy was relaxed. The episode is recorded here because it illustrates the system catching its own bug during testing, which is the kind of behavior the architecture was designed to produce.

12. Ethical Considerations and Limitations

12.1 Copyright and Intellectual Property

All data in the RAG knowledge base is drawn from openly licensed materials. OWASP content is used under Creative Commons Attribution-ShareAlike. CWE is public domain. Style guide excerpts are either public documents or summaries authored for this project. No proprietary code or corporate documents are included.

12.2 Bias and Representation

The system is evaluated on three languages (Python, JavaScript, Java) and on CWE categories that skew toward web application and backend concerns. It is likely weaker on systems languages such as C or Rust, on embedded contexts, and on domains such as smart contracts that have their own vulnerability taxonomies. These limitations are documented openly rather than papered over.

12.3 Misuse Scenarios

A code review system capable of identifying vulnerabilities is also, in principle, capable of being used to find vulnerabilities in code the user does not own. The synthetic data pipeline explicitly avoids producing weaponized exploit code, and the system's outputs are framed as remediation-oriented findings rather than exploit instructions. Emitting a concrete fix alongside every finding is itself a defensive posture: the remediation is the intended consumable, not the exploit.

12.4 Limitations

Three limitations deserve explicit acknowledgment. The system depends on an LLM whose behavior is not fully deterministic. The evaluation suite at ten hand-labeled samples is modest; confidence intervals on the reported metrics are wide. The system is not a substitute for formal security review in regulated contexts such as payment systems or medical devices, where human accountability and traceable review processes are mandatory.

13. Future Work

Several extensions are planned. Expanding the evaluation suite to at least 100 hand-labeled samples would reduce confidence interval width to usable ranges. Adding a Performance Analyst agent and an Architectural Synthesizer agent, each with its own specialized knowledge base, would extend the system beyond security and quality review. Integration with a continuous integration platform would enable blocking pull requests on unresolved critical findings, moving from advisory to gating. Replacing REINFORCE with a Deep Q-Network over the complete agent graph would close the loop on treating the multi-agent system as a learnable object.

A broader context frames this work. Two weeks before the submission of this report, Anthropic disclosed Claude Mythos Preview and the associated Project Glasswing initiative, in which the same vendor that produces the underlying model used by this project deploys that class of model to find zero-day

vulnerabilities in critical software at industrial scale. The public description of Project Glasswing is instructive because its architectural pattern is structurally similar to the architectural pattern explored here: files or code units are ranked by likelihood of containing bugs before analysis, specialized agent instances analyze different units in parallel, and a final, independent agent pass is used to confirm which findings are real and interesting before any report is surfaced to a human. That pattern, at Project Glasswing scale, is finding thousands of zero-day vulnerabilities in operating systems and web browsers. At the scale of this project, the same pattern, with an adversarial Evaluator substituted for the final confirmation agent and a curated RAG corpus substituted for autonomous code execution, is finding textbook vulnerabilities on a ten-sample suite and doing so with a measurable architecture-attributable improvement over a single-prompt baseline on the identical model. CodeSentinel does not compete with Project Glasswing on capability. It does demonstrate, within the bounds of an academic project, that the architectural pattern in use at the frontier is reproducible, testable, and teachable with open tools, and that its gains are attributable to the architecture rather than to the model underneath. Future work would involve substituting a stronger model for the current Claude Sonnet 4.5 backbone, extending the corpus beyond OWASP and CWE into language-specific weakness taxonomies, and exploring a discovery mode, modeled loosely on Project Glasswing, in which the system can optionally execute code in a sandbox to confirm hypothesized vulnerabilities before surfacing them. The architecture itself does not need to change to accommodate any of these; that is the point of the pattern.

14. Conclusion

CodeSentinel demonstrates that a small, disciplined multi-agent generative AI system can outperform a single-prompt baseline on code review. The three contributions driving this result are specialization via agent roles, grounding via retrieval-augmented generation against authoritative sources, and adversarial self-critique via a dedicated Evaluator Guardian with both programmatic and LLM validation. The project additionally contributes a synthetic data generation pipeline with independent verification, and a reinforcement learning enhancement layer that treats prompt and routing decisions as learnable policies. On the ten-sample hand-labeled toy suite, the multi-agent architecture achieves true positive rate 1.000 against baseline 0.750, with zero false positives and perfect CWE classification on both systems. All 27 unit tests pass on a clean clone. The underlying claim is broader than the specific system: when an LLM application is rearchitected into specialized, grounded, adversarially-reviewed agents, the gains come from the architecture, not from the model, and they compose with any model the field produces next.

15. References

OWASP Foundation. OWASP Top 10 2025. https://owasp.org/Top10/

MITRE. Common Weakness Enumeration (CWE). https://cwe.mitre.org/

Chase, H. LangChain and LangGraph documentation. https://python.langchain.com/docs/langgraph

Reimers, N., Gurevych, I. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP 2019.

Lewis, P. et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020.

Auer, P., Cesa-Bianchi, N., Fischer, P. Finite-time Analysis of the Multiarmed Bandit Problem. Machine Learning 47, 2002.

Williams, R. J. Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning. Machine Learning 8, 1992.

van Rossum, G., Warsaw, B., Coghlan, A. PEP 8: Style Guide for Python Code.

Google. Google Python Style Guide. https://google.github.io/styleguide/pyguide.html

Airbnb. Airbnb JavaScript Style Guide. https://github.com/airbnb/javascript

Anthropic. Project Glasswing: Securing Critical Software for the AI Era. April 2026. https://www.anthropic.com/glasswing

Anthropic Frontier Red Team. Claude Mythos Preview: Technical Notes on Vulnerability Discovery. April 2026. https://red.anthropic.com/2026/mythos-preview/

Appendix A: Prompt Template Excerpts

A.1 Security Sentinel System Prompt (excerpt)

```
You are a senior application security engineer.
Your only job is to identify security vulnerabilities.

Rules:
1. Every finding MUST cite a retrieved passage in
   rag_source. Findings without a citation are invalid.
2. If you are uncertain whether code is vulnerable,
   suppress the finding rather than invent one.
3. Output valid JSON matching the SecurityFinding schema.
```

A.2 Evaluator Guardian System Prompt (excerpt)

```
You are an adversarial reviewer.
Assume the upstream agents have made mistakes.

Reject any finding that meets ANY of these:
  - rag_source empty or not in retrieved context
  - evidence does not point to actual code lines
  - fix is vague or not implementable
  - schema is malformed
  - two findings contradict each other
```

Appendix B: Per-Sample Benchmark Results

The following table shows per-sample outcomes on the ten hand-labeled samples. All results are from a single deterministic mock-mode run of python -m eval.run_benchmark.

Sample	Language	Truth	Baseline TP	Baseline FP	Multi TP	Multi FP
TOY-001	Python	SQL	1	0	1	0
TOY-002	Python	Clean	0	0	0	0
TOY-003	Python	Pickle	1	0	1	0
TOY-004	Python	CmdInj	1	0	1	0
TOY-005	Python	MD5	0	0	1	0
TOY-006	Python	TLS	1	0	1	0
TOY-007	Python	Clean	0	0	0	0
TOY-008	JS	SQL	1	0	1	0
TOY-009	JS	XSS	1	0	1	0
TOY-010	Python	yaml	0	0	1	0

Totals			6	0	7	0

The two samples on which the systems diverge are TOY-005 (hashlib.md5 for password hashing) and TOY-010 (yaml.load without SafeLoader). Both require recognizing a language-specific pattern that the baseline single prompt does not reliably surface. The multi-agent system's RAG retrieval step supplies the PY-07 (yaml.load without SafeLoader) and PY-08 (Python MD5 or SHA1 for security) patterns directly to the Security Sentinel, which is why the multi-agent catches both.